

CPP Module 02 — Correction Guide

Sep 29, 2026 · @Bruno

A walkthrough of the Fixed-point class and BSP, in the order an evaluator checks them: what each piece does, why it is written that way, and the concept behind it.

Defense flow

Run the correction in this order: global checks first, then one exercise at a time, showing the output before explaining the code.

1. **Clone and build.** In each `exNN/`, run `make`, then `make` again. The second call must print `Nothing to be done` (no relink). Then `make fclean` to show the binary and `.o` files are removed.
2. **Flags.** Show the Makefile uses `c++ -Wall -Wextra -Werror -std=c++98`.
3. **Forbidden things.** `grep -rn "using namespace\|friend" .` returns nothing. No STL containers, no `<algorithm>`, no `printf/malloc`.
4. **Headers.** Every `.hpp` has include guards, contains declarations only (no function bodies), and compiles on its own.
5. **Per exercise.** Run the binary, compare with the subject's expected output, then walk through the class: OCF members first, then the new functions of that exercise.
6. **Leaks.** `valgrind ./fixed` shows no leaks. Nothing is allocated with `new`, so there is nothing to leak.

```
make && make          # 2nd call: "Nothing to be done for 'all'"
./fixed               # compare with the subject
valgrind --leak-check=full ./fixed
make fclean
```

New concepts in Module 02

Eight ideas carry the whole module. Know each one in a sentence and an example.

Fixed-point numbers

A real number stored in an `int` by scaling it: `raw = value x 2^8 = value x 256`. The low 8 bits hold the fraction, so the smallest step is $1/256 = 0.00390625$.

```
int (32 bits)    [ 24 bits: integer part | 8 bits: fraction ]
42.42  ->  42.42 x 256 = 10859.52  ->  round  ->  10860
10860  ->  10860 / 256 = 42.421875  ->  prints 42.4219
```

Why it exists: integer arithmetic is fast and exact on hardware without an FPU, with a fixed, predictable precision (graphics, audio, embedded). Floats have more range but variable precision.

Orthodox Canonical Form (OCF)

From Module 02 on, every class declares these 4 members explicitly:

Member	Signature	Called when
Default constructor	<code>Fixed()</code>	<code>Fixed a;</code>
Copy constructor	<code>Fixed(const Fixed &src)</code>	a NEW object is built from another: <code>Fixed b(a);</code> , <code>Fixed b = a;</code> , pass or return by value
Copy assignment	<code>Fixed &operator=(const Fixed &src)</code>	an EXISTING object is overwritten: <code>b = a;</code>
Destructor	<code>~Fixed()</code>	the object goes out of scope or is deleted

The compiler generates them if you don't, but the generated copy is shallow. Writing them makes the copy behavior explicit, which matters as soon as a class owns memory (Module 04's `Brain*`).

Operator overloading (ad-hoc polymorphism)

Ad-hoc polymorphism means one name, several implementations chosen by argument types at compile time. `a + b` on two `Fixed` calls `Fixed::operator+`, on two `int` the built-in `+`. Two ways to overload:

- **Member:** the left operand is `*this`. `a > b` becomes `a.operator>(b)`. Used for all comparison, arithmetic and increment operators.

- **Non-member:** needed when the left operand is not your class. `std::cout << a` becomes `operator<<(std::cout, a)`, and you cannot add methods to `std::ostream`.

Rule of thumb: make the operator behave like it does for `int` (same return type, same chaining).

const correctness

- `int toInt(void) const;` a const method promises not to modify `*this`, so it can be called on const objects (`Fixed const b(10); b.toInt();`).
- `Fixed const &rhs` a const reference parameter: no copy, and the function cannot modify the argument.
- `Fixed const x;` a const member: set once in the initializer list, never reassigned (ex03).

static members

- `static const int _fractionalBits = 8;` one value shared by every instance, not stored in each object.
- `static Fixed &max(Fixed &a, Fixed &b);` a function of the class with no `this`, called as `Fixed::max(a, b)`.

References vs copies

Return type	Meaning	Used by
<code>Fixed &</code>	the same object, no copy	<code>operator=</code> , <code>++a</code> , <code>min / max</code>
<code>Fixed</code>	a new object (copy)	<code>+</code> <code>-</code> <code>*</code> <code>/</code> , <code>a++</code>
<code>std::ostream &</code>	the same stream, for chaining	<code>operator<<</code>

Return a reference only to something that outlives the call (`*this`, an argument). Never to a local variable: it dies when the function returns.

Initializer lists

`Fixed::Fixed() : _value(0) {}` constructs the member directly with its value. Assigning in the body (`{ _value = 0; }`) constructs first, then assigns. For `const` members and references, the initializer list is the only option.

Encapsulation and the `friend` ban

Private data is reached only through a public interface the class chooses (`toFloat()`, `getX()`). `friend` would let an outside function bypass `private`, so 42 forbids it (-42). `operator<<` and `bsp()` stay outside the class and use public methods instead.

ex00: My first class in OCF

Goal: a `Fixed` class that only stores raw bits, with the 4 OCF members tracing every call.

Class layout

```
class Fixed {
private:
    int            _value;           // raw bits
    static const int _fractionalBits = 8; // shared, constant
public:
    Fixed();
    Fixed(const Fixed &src);
    Fixed &operator=(const Fixed &src);
    ~Fixed();
    int    getRawBits(void) const;
    void    setRawBits(int const raw);
};
```

Logic of each member

- **Default constructor** : `_value(0)` sets the value to 0 through the initializer list.
- **Copy constructor** : `_value(src._value)` copies directly. The subject's version calls `*this = src;` inside it, which adds a `Copy assignment operator` called `line`. Mine doesn't, and the subject marks that line as optional.
- **operator=** does 3 things: `if (this != &src)` compares addresses to skip `a = a;` copies `_value;` `return (*this);` returns the object itself by reference, so `a = b = c` works as `a = (b = c)`.
- **Destructor** only prints: nothing was allocated.

- `getRawBits` is `const` : reading does not modify the object.

Tracing the subject's main

Line	Function called	Why
<code>Fixed a;</code>	default ctor	new object, no argument
<code>Fixed b(a);</code>	copy ctor	new object from <code>a</code>
<code>Fixed c;</code>	default ctor	new object
<code>c = b;</code>	<code>operator=</code>	<code>c</code> already exists
end of <code>main</code>	destructor x3	<code>c</code> , <code>b</code> , <code>a</code> , in reverse order of creation

Why self-assignment matters, even if harmless here: with a class that owns heap memory, `operator=` does `delete[] _data;` then copies from `src._data`. If `src` is `*this`, it copies from freed memory.

ex01: Conversions and operator<<

Goal: build a `Fixed` from an `int` or a `float`, convert it back, and print it with `<<`. Every conversion is a multiply or divide by 256.

```

            x 256 (and round)
int / float -----> raw bits
            <-----
            / 256 or >> 8

```

Logic of each function

Function	Code	Worked example
int ctor	<code>_value = n * (1 << 8);</code>	<code>10 -> 2560</code>
float ctor	<code>_value = roundf(n * (1 << 8));</code>	<code>42.42 x 256 = 10859.52 -> 10860</code>
<code>toInt</code>	<code>return _value >> 8;</code>	<code>10860 >> 8 = 42</code>
<code>toFloat</code>	<code>return _value / (float)(1 << 8);</code>	<code>10860 / 256.0 = 42.4219</code>

Decisions to explain

- `n * 256`, not `n << 8`. Left-shifting a negative `int` is undefined behavior in C++98. Multiplication gives the same result for positives and is defined for negatives.
- `roundf`, not a cast. A cast truncates: 10859.52 would become 10859 (42.4180). `roundf` gives the nearest representable value, 10860 (42.4219). It is the only allowed function.
- Divide as a `float` in `toFloat`. `10860 / 256` in integers gives 42 and loses the fraction; casting the divisor to `float` keeps it.
- Why 42.42 prints 42.4219: only multiples of 1/256 are representable. 42.4219 is the closest one.

`operator<<` : a non-member function

```
std::ostream &operator<<(std::ostream &o, Fixed const &f) {
    o << f.toFloat();    // public method: no friend needed
    return (o);          // same stream, for chaining
}
```

- It cannot be a member of `Fixed`: in `cout << a`, the left operand is `std::ostream`.
- `std::cout << a << b` is evaluated as `(std::cout << a) << b`. The first call prints `a` and returns `std::cout`; the second prints `b`. Returning `void` would break the chain.

Tracing `a = Fixed(1234.4321f);`

Float constructor called (temporary built), Copy assignment operator called (copied into existing `a`), Destructor called (temporary dies at the end of the statement).

ex02: Operator overloading

Goal: make `Fixed` behave like a number: compare, compute, increment, and pick min/max. The key question for each operator is what happens to the x256 scale.

Comparison: `>` `<` `>=` `<=` `==` `!=`

Both sides carry the same scale, so comparing raw values gives the same answer as comparing real values. `2.5 > 1.0` is `640 > 256`. Each returns `bool` and is `const`.

Arithmetic: what happens to the scale

Operator	Raw math	Scale after	Fix
+	$(a \times 256) + (b \times 256) = (a + b) \times 256$	kept	none, add raw values
-	$(a \times 256) - (b \times 256) = (a - b) \times 256$	kept	none, subtract raw values
*	$(a \times 256) \times (b \times 256) = (a \times b) \times 256 \times 256$	doubled	compute in float, convert back
/	$(a \times 256) / (b \times 256) = a / b$	lost	multiply numerator by 256 first

- **Why * goes through float:** $200 \times 256 = 51200$, and 51200×51200 overflows an int (max about 2.1 billion), so $200 * 200$ printed -25536. `Fixed(a.toFloat() * b.toFloat())` gives 40000.
- **Why / multiplies first:** $(2560 \times 256) / 1024 = 640 = 2.5$. Dividing first would give $2560 / 1024 = 2$ raw, and the fraction is lost.
- **Division by zero:** not handled. The subject says a crash is acceptable.
- All four return a new `Fixed` by value: `a + b` must not modify `a` or `b`, so there is no existing object to reference.

Increment and decrement

The step is epsilon, 1 raw unit = $1/256$, the smallest `e` with $1 + e > 1$. That is why `++a` from 0 prints 0.00390625.

```
Fixed &Fixed::operator++() {           // ++a (pre)
    ++_value;
    return (*this);                   // the object itself, already changed
}

Fixed Fixed::operator++(int) {         // a++ (post): dummy int = postfix marker
    Fixed temp(*this);                // save old value (copy ctor)
    ++_value;
    return (temp);                    // return OLD value by copy
}
```

Expression	Prints	a after
<code>++a (a = 0)</code>	0.00390625	0.00390625
<code>a++ (a = 0.00390625)</code>	0.00390625	0.0078125

Post-increment cannot return a reference: `temp` is a local and dies at the end of the call.

`min / max` : static, with const overloads

- `static` : they compare two objects and belong to neither, so there is no `this` . Called as `Fixed::max(a, b)` .
- Return a reference to one of the arguments: no copy, and it outlives the call.
- Two overloads: in the subject's main, `b` is `Fixed const` . A `Fixed &` parameter cannot bind to a const object, so `Fixed::max(a, b)` picks the `Fixed const &` version.

Subject main result: `b = 5.05 x 2 . 5.05 x 256 = 1292.8 -> 1293` , and `1293 / 256 x 2 = 10.1016` , not 10.1: rounding error from the 1/256 precision.

ex03: BSP (point in triangle)

Goal: `bsp(a, b, c, point)` returns true only if `point` is strictly inside triangle `abc` ; on an edge or a vertex it returns false. This exercise is optional: the module passes without it.

The `Point` class

- Two private members `Fixed const x;` and `Fixed const y;` : a point never moves once built.
- `const` members must be set in the initializer list: `Point(float x, float y) : x(x), y(y) {}` . Assigning in the body does not compile.
- **`operator=` does nothing and returns `*this`** . It is required by the OCF, but const members can never be reassigned. After `q = p` , `q` keeps its own values. This is the expected answer, not a bug.
- Getters `Fixed const &getX() const` : return a const reference, so there is no copy and the caller can only read. That is how `bsp()` , a free function, reads private coordinates without `friend` .

The algorithm: cross product sign

For an edge `p1 -> p2` and a point `p`, the 2D cross product tells which side of the line `p` is on:

```
cross = (p2.x - p1.x) * (p.y - p1.y) - (p2.y - p1.y) * (p.x - p1.x)

      p   cross > 0  (left)
      .
p1 -----> p2      cross == 0  (on the line)
      .
      p   cross < 0  (right)
```

Walk the triangle `a -> b -> c -> a` and compute the sign for each edge:

1. `d1 = cross(a, b, p)`, `d2 = cross(b, c, p)`, `d3 = cross(c, a, p)`.
2. If any of them is 0, the point is on an edge or a vertex: return false.
3. If all three have the same sign, the point is on the same side of every edge: inside, return true.
4. Mixed signs: outside, return false.

Checking same sign, not all positive, makes it work for vertices given clockwise or counter-clockwise.

Worked example (triangle `a(0,0)`, `b(4,0)`, `c(0,4)`)

Point	d1 (a->b)	d2 (b->c)	d3 (c->a)	Result
(1,1)	4	8	4	true, all positive
(10,10)	40	-64	40	false, mixed
(2,0)	0	8	8	false, on edge a->b

Other decisions

- `crossSign` is `static` at file level: visible only inside `bsp.cpp`. Its `result` is a plain local variable, not `static` (a static local would persist between calls for no reason).
- `bsp` takes `Point` `const` by value because the subject's prototype says so. Each argument calls the `Point` copy constructor.
- The `Fixed` class is reused from `ex02` with the trace messages removed, so the test output stays readable.
- `main.cpp` tests 5 cases: inside, outside, on a vertex, on an edge, just inside at (0.1, 0.1).

Likely evaluator questions

Question	Short answer
Copy constructor or <code>operator=</code> : which one runs in <code>Fixed b = a;</code> ?	Copy constructor: <code>b</code> is being created. The <code>=</code> in a declaration is initialization.
Why check <code>this != &src</code> ?	To skip self-assignment. Harmless for an <code>int</code> , but a class owning heap memory would free its data, then copy from the freed memory.
Why does <code>operator=</code> return <code>Fixed &</code> ?	To allow <code>a = b = c</code> without an extra copy, like with <code>int</code> .
Why is <code>_fractionalBits</code> <code>static</code> ?	One shared value for all instances, not stored in each object.
Why is <code>operator<<</code> not a member?	Its left operand is <code>std::ostream</code> , not <code>Fixed</code> .
How does it read <code>_value</code> without <code>friend</code> ?	Through the public <code>toFloat()</code> .
Why <code>roundf</code> in the float constructor?	A cast truncates; <code>roundf</code> picks the nearest representable value.
Why does 42.42 print 42.4219?	Only multiples of 1/256 are representable; 10860/256 is the closest.
Why do comparisons use raw values?	Both sides have the same x256 scale, so the order is the same.
What goes wrong with raw <code>*</code> and <code>/</code> ?	<code>*</code> doubles the scale (and overflows); <code>/</code> loses it.
Difference between <code>++a</code> and <code>a++</code> ?	<code>++a</code> returns the changed object by reference; <code>a++</code> returns a copy of the old value. The <code>int</code> parameter only marks the postfix version.
What is epsilon?	1 raw unit = 1/256 = 0.00390625, the smallest step.
Why are <code>min / max</code> <code>static</code> ?	They need no <code>this</code> ; called as <code>Fixed::max(a, b)</code> .

Question	Short answer
Why a const and a non-const overload?	A <code>Fixed &</code> cannot bind to a const object like <code>b</code> in main.
Why can't <code>Point::operator=</code> copy?	<code>x</code> and <code>y</code> are <code>const</code> : set once in the initializer list, never reassigned.
Why does bsp return false on an edge?	The subject requires it; a cross product of 0 means the point is on the line.
Why check same sign rather than positive?	So the result doesn't depend on the vertex order (clockwise or not).
What is ad-hoc polymorphism?	Same function or operator name, different implementations chosen at compile time from the argument types (overloading).